



Dead Harvest
Fase de Análisis - Parte 1

Campo	Detalle
Proyecto	Dead Harvest - Videojuego 2D Supervivencia Cenital
Entregable	Fase de Análisis Parte 1: Resumen, Justificación, Objetivos, Requerimientos y Casos de Uso
Equipo	Iron Pixels
Versión	0400 - Fase Análisis
Fecha	11/08/2026
Responsables	Cristian Sahón (Resumen y Objetivos) - Gilmar Maldonado (Justificación) - Marcos Sulugüí (Integración y ERS) - Hefer Cotúc (Casos de Uso)
GitHub	https://github.com/Crissssss2401/iron-pixels-is2.git
Metodología	Scrum - ERS 29 páginas, MoSCoW, UML

ÍNDICE

1. Resumen del Sistema
2. Justificación del Proyecto
 - 2.1 ¿Qué problema busca resolver?
 - 2.2 ¿Por qué es relevante?
 - 2.3 ¿Qué aporta la Ingeniería de Software?
 - 2.4 Importancia Técnica
 - 2.5 Importancia Académica y Social
3. Objetivos del Proyecto
 - 3.1 Objetivo General
 - 3.2 Objetivos Específicos
4. Integración de Requerimientos con formato ERS
 - 4.1 Requisitos Generales
 - 4.2 Requisitos Funcionales (8 RF Corregidos)
 - 4.3 Requisitos No Funcionales (3 RNF Corregidos)
 - 4.4 Reglas de Negocio y Trazabilidad
5. Casos de Uso - Inicio Fase de Diseño
 - 5.1 Diagrama de Casos de Uso
 - 5.2 Especificación CU-01 Atacar
6. Inicio Fase de Diseño - Próximos Diagramas de Comportamiento
7. Conclusiones

1. Resumen del Sistema

Dead Harvest es un videojuego de acción y supervivencia en 2D con perspectiva cenital, en el que el jugador comienza atrapado dentro de una zona de cuarentena completamente infestada después de una evacuación fallida. Para escapar, deberá sobrevivir a sucesivas oleadas de zombis utilizando armas cuerpo a cuerpo, administrando su vida y estamina, recolectando recursos (botiquines) y aprovechando mejoras que le permitan enfrentar enemigos cada vez más peligrosos.

Conforme avanza la partida, las oleadas aumentan progresivamente su dificultad e incorporan diferentes tipos de zombis (Caminante, Corredor, Acorazado, Explosivo) que obligan al jugador a adaptar su estrategia de combate. La experiencia concluye cuando el jugador supera las oleadas establecidas, derrota al jefe final y logra escapar de la zona de cuarentena; si pierde toda su vida antes de conseguirlo, la partida finaliza en derrota y se muestra la pantalla de resultados con tiempo sobrevivido, enemigos eliminados y oleada alcanzada.

Flujo principal concreto: Menú Principal -> Nueva Partida -> Arena (Oleada N) -> Combate melee (Q/E cambio arma, clic ataque, dash esquiva, gestión estamina) -> Recolección botiquines -> Oleada superada -> Elección mejora -> Siguierte oleada (N+1 más difícil) -> Jefe Final (Oleada 10) -> Escape / Victoria o Derrota -> Pantalla Resultados -> Jugar de nuevo / Menú.

2. Justificación del Proyecto

2.1 ¿Qué problema busca resolver?

Dead Harvest busca ofrecer una experiencia de entretenimiento basada en partidas rápidas (15-20 min), desafiantes y dinámicas, donde el jugador deba tomar decisiones constantemente para sobrevivir. Resuelve la necesidad de contar con una experiencia sencilla de comprender pero con desafío progresivo. Utiliza mecánicas de movimiento 8 direcciones, esquiva, combate cuerpo a cuerpo exclusivo, administración de estamina, enfrentamiento contra 4 tipos de enemigos y mejoras entre oleadas. El jugador no solo reacciona, sino que administra recursos y decide cuándo atacar, esquivar o aprovechar mejoras.

2.2 ¿Por qué el proyecto resulta relevante?

Es relevante porque combina diferentes elementos de programación y diseño dentro de un mismo sistema: partidas basadas en oleadas con dificultad progresiva, combate cuerpo a cuerpo que obliga a acercarse y calcular timing, y sistemas interconectados. Permite aplicar conceptos de diseño de videojuegos como interacción jugador-sistema, inteligencia artificial (FSM), interfaz de usuario, progresión, administración de recursos y diseño de niveles, en un MVP viable para un equipo de 6 personas en tiempo académico.

2.3 ¿Qué aporta la Ingeniería de Software?

La Ingeniería de Software permite organizar y controlar el desarrollo mediante procesos, técnicas y herramientas. El uso de Scrum divide el desarrollo en Sprints (Sprint 1: #1 Movimiento, #2 Ataque, #3 IA básica, #4 Oleadas, #5 HUD), facilita planificación y asignación. Herramientas como ERS (plantilla 29 páginas), historias de usuario (14 HU #1-#17), casos de uso (CU-01 a CU-09), diagramas UML, Product Backlog MoSCoW y GitHub Projects con columnas Backlog RF, RNF, Backlog, Sprint 1, Roadmap permiten definir claramente funcionalidades antes de implementar, reducir errores, evitar inconsistencias y mantener trazabilidad requisitos -> código.

2.4 Importancia Técnica

Técnicamente integra: movimiento y control, combate melee con hitbox, daño y vida, estamina, 4 tipos de enemigos, IA por FSM (Patrullar, Perseguir, Atacar), sistema de oleadas, mejoras y progresión, HUD, menú principal, pantalla resultados e interacción jugador-sistemas. Requiere establecer relaciones entre objetos y sistemas. El análisis, diseño y documentación con IS son clave para arquitectura organizada, uso de pooling para 50 enemigos a 60 FPS (RNF-01 #25), respuesta <100ms (RNF-03 #21) y usabilidad 90% en 60s (RNF-02 #14), facilitando mantenimiento y escalabilidad (añadir nuevo tipo zombi en <2h).

2.5 Importancia Académica y Social

Académicamente aplica requisitos funcionales/no funcionales, HU, CU, diagramas UML, Scrum, Product Backlog, documentación técnica y control GitHub. Cada integrante asume rol específico (Scrum Master, Documentación, Herramientas, Comunicación, UI/UX, QA/Product Owner), experimentando proceso real. Socialmente resuelve necesidad de diversión accesible, partidas rápidas que fomentan pensamiento estratégico, gestión de recursos y toma de decisiones bajo presión, sin mecánicas adictivas predatorias (sin microtransacciones, fuera de alcance v1.0).

3. Objetivos del Proyecto

3.1 Objetivo General

Desarrollar un videojuego de acción y supervivencia en 2D con perspectiva cenital mediante Unity 2022.3 LTS, implementando mecánicas de combate cuerpo a cuerpo, gestión de recursos (vida, estamina), progresión por oleadas e inteligencia artificial para enemigos con FSM, con el propósito de ofrecer una experiencia de juego dinámica, estratégica y desafiante que pueda completarse en aproximadamente veinte minutos y que sirva como MVP evaluable de la aplicación de Ingeniería de Software y Scrum.

3.2 Objetivos Específicos

#	Qué voy a hacer?	Cómo lo voy a hacer?	Para qué lo voy a hacer?	RF/RNF
OE-01	Diseñar e implementar sistema de combate cuerpo a cuerpo con 4 armas (espada, hacha, mazo, lanza)	Mediante animaciones, hitbox, cálculo de daño, alcance y velocidad diferenciada por arma en Unity Animator y C#	Para permitir que el jugador adapte estrategia de combate según enemigos y situaciones durante partida	RF-01 #18, RF-02 #19, RF-08 #20
OE-02	Implementar IA para enemigos con comportamientos diferenciados	Mediante máquinas de estados finitos (FSM) con estados Patrullar, Perseguir, Atacar, reacción a daño, y 4 tipos (Caminante, Corredor, Acorazado, Explosivo)	Para generar amenaza constante y variedad en enfrentamientos	RF-03 #23 CORREGIDO
OE-03	Desarrollar sistema progresivo de oleadas	Mediante generación controlada con spawner, conteo por tipo, incremento $N+1 > N$ y pausa breve, incluyendo jefe final en oleada 10	Para mantener desafío durante toda partida y proporcionar progresión clara hasta enfrentamiento final	RF-04 #24 CORREGIDO
OE-04	Implementar sistemas de gestión de vida y estamina	Mediante reglas RN-01 a RN-04: estamina nunca negativa, reducción al atacar/esquivar, penalización	Para evitar combate basado en ataques continuos y fomentar administración	RF-01, RF-08, RN-01 a RN-04

		50% si 0, botiquín +25% sin superar máximo	estratégica de recursos	
OE-05	Diseñar interfaz de usuario clara y funcional	Mediante HUD con vida, estamina, arma equipada y oleada en tiempo real, menú principal y pantalla resultados con UGUI, mínima invasión	Para permitir que jugador conozca estado y tome decisiones sin interrumpir experiencia	RF-05 #11, RF-06 #12, RF-07 #13, RNF-02 #14
OE-06	Evaluar y ajustar jugabilidad y rendimiento	Mediante pruebas funcionales (ataque, oleadas), usabilidad (90% en 60s) y rendimiento (50 zombis a 60 FPS) durante desarrollo, control GitHub	Para obtener versión estable, comprensible y equilibrada del MVP y demostrar aplicación de IS	RNF-01 #25, RNF-03 #21, OE-01 a OE-05

4. Integración de Requerimientos con formato ERS trabajado

Se integra el catálogo corregido y alineado a GitHub (v0200) que supera auditoría de trazabilidad. Estado 100% alineado tras mover Issues #23 y #24 de columna RNF a RF.

4.1 Requisitos Generales

ID	Nombre	Descripción	Dependencias
RG-01	Sistema de Combate Cuerpo a Cuerpo	Combate melee con 4 armas diferenciadas.	ON-02
RG-02	Sistema de IA de Enemigos	IA con comportamientos diferenciados por tipo. RF-03 #23 CORREGIDO.	ON-01

RG-03	Sistema de Oleadas y Progresión	Gestión oleadas con dificultad incremental. RF-04 #24 CORREGIDO.	ON-01
RG-04	Sistema de Interfaz y Experiencia	HUD, menú principal y resultados de forma clara.	ON-01

4.2 Requisitos Funcionales (8 RF Corregidos - Backlog RF GitHub)

ID GitHub	Nombre	Descripción RU/RS	Estado
RF-01 #18	Ataque cuerpo a cuerpo	RU: Como jugador, quiero atacar cuerpo a cuerpo. RS: Animación al clic, hitbox, daño arma, VFX/SFX.	OK
RF-02 #19	Cambio de armamento	RU: Cambiar armas Q/E. RS: Q/E cambia, stats inmediatos, 4 armas.	OK
RF-03 #23 CORREGIDO	Inteligencia Artificial Enemigos	RU: Zombis creíbles. RS: FSM Patrullar/Perseguir/Atacar. CORRECCIÓN: Estaba en RNF por error.	CORREGIDO
RF-04 #24 CORREGIDO	Sistema De Oleadas	RU: Oleadas crecientes. RS: Coordinar cantidad, incrementar dificultad. CORRECCIÓN: Estaba en RNF por error.	CORREGIDO
RF-05 #11	HUD	RU: Ver estado sin interrumpir. RS: Vida, estamina, oleada, arma tiempo real.	OK
RF-06 #12	Menú Principal	RU: Acceder a menú. RS: Nueva	OK

		Partida, Opciones, Salir.	
RF-07 #13	Pantalla de Resultados	RU: Ver resumen. RS: Victoria/derrota, tiempo, kills, oleada, rejugar/menú.	OK
RF-08 #20	Sistema de estamina	RU: Gestión estratégica. RS: Reduce al atacar/esquivar, regenera, penaliza si 0.	OK

4.3 Requisitos No Funcionales (3 RNF Corregidos - Backlog RNF GitHub)

ID	Nombre	Descripción Medible	Estado
RNF-01 #25	Rendimiento	Soportar 50 zombis sin bajar de 60 FPS. Métrica: FPS promedio >60 con 50 entidades.	OK
RNF-02 #14	Usabilidad	90% jugadores inician partida e identifican vida, estamina, oleada en 60s sin ayuda.	OK
RNF-03 #21	Tiempo de respuesta	Entrada jugador -> reflejo pantalla <100ms. Crítica.	OK

NOTA: Se eliminan RNF-04 Fiabilidad, RNF-05 Portabilidad, RNF-06 Mantenibilidad de versión 0101 porque NO existen en GitHub. ERS queda 100% alineada.

4.4 Reglas de Negocio y Trazabilidad

Código	Regla	Trazabilidad
RN-01	Estamina nunca negativa ni > máximo.	RF-08
RN-02	Si vida <=0, partida derrota.	RF-01, CU-01 FE-02

RN-03	Si estamina=0, velocidad ataque/movimiento -50% hasta 20%.	RF-01, RF-08
RN-04	Botiquín no cura por encima de vida máxima.	RF-01
RN-05	Oleada N+1 > N enemigos.	RF-04 #24

5. Casos de Uso - Inicio Fase de Diseño

5.1 Diagrama de Casos de Uso

Actor Jugador -> CU-01 Atacar, CU-02 Cambiar arma, CU-03 Esquivar, CU-04 Recoger botiquín, CU-05 Iniciar partida, CU-08 Menú, CU-09 Resultados. Actor Sistema Juego -> CU-06 Administrar oleadas, CU-07 Visualizar HUD. Relaciones: CU-01 <<include>> CU-03 (estamina), CU-01 <<include>> CU-07 (HUD), CU-05 <<include>> CU-07, CU-06 <<extend>> CU-04 (botiquín opcional), CU-Finalizar <<extend>> CU-09 (resultados condicional). Diagrama editable: [diagramas/Diagrama_Casos_Uso.drawio.png](#) (ver ERS_Semana04).

5.2 Especificación CU-01 Atacar

Campo	Detalle
Código	CU-01 Atacar cuerpo a cuerpo - Responsable Cristian Sahón
Objetivo	Permitir que jugador ejecute ataque con arma equipada para eliminar enemigos.
Actor Principal	Jugador
Actor Secundario	Sistema del Juego
Precondiciones	Partida iniciada, arma equipada, estamina suficiente, jugador vivo, enemigo activo.
Flujo Principal	1. Jugador presiona ataque 2. Sistema verifica arma 3. Verifica estamina 4. Animación arma 5. Activa hitbox 6. Detecta colisión 7. Calcula daño 8. Reduce vida enemigo 9. Elimina si 0 10. VFX/SFX 11. Actualiza HUD 12. Fin
Alternativos	FA-01 Sin enemigo en alcance: animación + consume estamina sin daño. FA-02 Enemigo sobrevive: continúa IA.
Excepciones	FE-01 Estamina insuficiente: cancela ataque. FE-02 Jugador derrotado: fin partida -> resultados.
Postcondiciones	Estamina disminuye, vida enemigo actualizada, eliminado si 0, HUD refleja cambios, listo para nueva acción.

RF	RF-01 #18, RF-08 #20, RN-01, RN-03, RN-04, HU-02 #2, Must Have Sprint 1
-----------	---

6. Inicio Fase de Diseño - Próximos Diagramas de Comportamiento

Con esta Fase de Análisis Parte 1 se cierra análisis y se inicia Fase de Diseño Parte 1 solicitada por el ingeniero:

Diagrama Comportamiento	Estado	Responsable Sugerido	Fuente
Diagrama de caso de uso	Hecho - diagramas/Diagrama Casos Uso.png	Efer Cotúc	ERS_Semana04 Cap 2
Escenario de caso de uso (CU-01)	Hecho - Cap 3 Flujo principal/alternativo /excepción	Cristian Sahón	Este documento Cap 5.2
Diagrama de actividades (Flujo de Partida PNI-01)	Pendiente - Debe modelar: Menú -> Nueva Partida -> Oleada N -> Combate -> Botiquín -> Mejora -> N+1 -> Jefe -> Resultados	Por asignar - Recomendado Nahum o Wendy	PNI-01
Diagrama de secuencia o comunicación (CU-01 Atacar)	Pendiente - Secuencia: Jugador -> Sistema -> Verificación estamina -> Animación -> Hitbox -> Enemigo -> Daño -> HUD	Por asignar - Recomendado Cristian o Efer	CU-01
Diagrama de estado (Enemigo FSM o Jugador)	Pendiente - Estados: Patrullar, Perseguir, Atacar, Herido, Muerto / Jugador: Vivo, Atacando, Sin estamina, Muerto	Por asignar - Recomendado Gilmar o Nahum	RF-03 #23

Integración: Los diagramas de comportamiento deben mantener trazabilidad con RF-01 a RF-08 y RNF-01 a RNF-03 corregidos, y con RN-01 a RN-05.

7. Conclusiones

La Fase de Análisis Parte 1 define de forma concreta el resumen del sistema (inicio en cuarentena infestada, fin escape tras oleadas + jefe o derrota), justifica el proyecto técnica (integración sistemas, 60 FPS, <100ms, pooling), académica (aplicación Scrum, ERS, UML, GitHub) y socialmente (diversión estratégica sin monetización predatoria), establece 1 objetivo general y 6 específicos con qué/cómo/para qué, integra 8 RF y 3 RNF corregidos alineados a GitHub Issues #1-#25, y documenta casos de uso base para iniciar Fase de Diseño de diagramas de comportamiento. Queda listo para entrega del 11/08/2026 23:59.